```
# ====================================================
# NAIVE BAYES CLASSIFICATION ON IRIS DATASET
# ====================================================

# Step 1: Import libraries
import pandas as pd
import numpy as np

# Step 2: Load dataset
df = pd.read_csv("/content/iris.csv")   # use correct path if needed

print("Dataset Loaded Successfully\n")

# Step 3: Display dataset
print("First 5 rows:\n", df.head())

# Step 4: Dataset info
print("\nDataset Info:")
print(df.info())

# Step 5: Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Step 6: Remove unnecessary column (Id)
if 'Id' in df.columns:
   df.drop('Id', axis=1, inplace=True)

# Step 7: Feature and target selection
X = df.iloc[:, :-1].values   # all columns except last
y = df.iloc[:, -1].values    # last column (Species)
```

```
# Step 8: Train-test split
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.2, random_state=0
)

# Step 9: Feature scaling
from sklearn.preprocessing import StandardScaler
sc = StandardScaler()
X_train = sc.fit_transform(X_train)
X_test = sc.transform(X_test)

# Step 10: Apply Naive Bayes
from sklearn.naive_bayes import GaussianNB
model = GaussianNB()
model.fit(X_train, y_train)

# Step 11: Prediction
y_pred = model.predict(X_test)

print("\nPredictions:\n", y_pred)

# ======================================================
# EVALUATION
# ======================================================

# Step 12: Confusion Matrix
from sklearn.metrics import confusion_matrix, accuracy_score, classification_report

cm = confusion_matrix(y_test, y_pred)
```

```
print("\nConfusion Matrix:\n", cm)

# Step 13: Accuracy
accuracy = accuracy_score(y_test, y_pred)
print("\nAccuracy:", accuracy)

# Step 14: Error Rate
error_rate = 1 - accuracy
print("Error Rate:", error_rate)

# Step 15: Precision & Recall (multi-class)
print("\nClassification Report (Precision, Recall, F1-score):")
print(classification_report(y_test, y_pred))

# ====================================================
# ADDITIONAL METRICS: TP, FP, TN, FN PER CLASS
# ====================================================

import numpy as np

classes = np.unique(y_test)

print("\n===== TP, FP, TN, FN FOR EACH CLASS =====")

for i, cls in enumerate(classes):
    TP = cm[i, i]
    FN = np.sum(cm[i, :]) - TP
    FP = np.sum(cm[:, i]) - TP
    TN = np.sum(cm) - (TP + FP + FN)

    print(f"\nClass: {cls}")
```

```
print("TP:", TP)
print("FP:", FP)
print("TN:", TN)
print("FN:", FN)

# Precision
precision = TP / (TP + FP) if (TP + FP) != 0 else 0
print("Precision:", precision)

# Recall
recall = TP / (TP + FN) if (TP + FN) != 0 else 0
print("Recall:", recall)
```